

SUMMER INTERNSHIP / PROJECT REPORT

Industrial Equipment Health Monitoring & Maintenance Dashboard

A Web-Based Software Engineering Prototype for a Thermal Power Plant

Submitted By	Anushka Bansal
Roll Number	23ucs543
Programme	B.Tech. Computer Science and Engineering
Project Type	Academic / Internship Software Project
Faculty / Industry Mentor	Dr. Aniket Pal

Abstract

This report explains the work I did on my project the Industrial Equipment Health Monitoring & Maintenance Dashboard. I worked as a Software Engineering intern and developer. I built a web-based monitoring application for a simulated power plant environment. My main task was to create a dashboard that helps engineers check the health of equipment. The dashboard allows engineers to see sensor readings manage faults and create reports.

During the project I focused on making an interface that's clean and easy to use. The interface shows the health of all the equipment in the plant at a glance. I used colour-coded status indicators updating sensor cards and interactive charts. I also worked on organizing the data that the dashboard uses. This data includes equipment, sensors, faults, maintenance tasks and alerts. I made sure that the dashboard, the equipment list and the reports all match each other. I also tested the application to find any problems with how the information was displayed.

This project was a learning opportunity. It showed me how a real monitoring dashboard is built and organized. I learned about data modeling, visualization and reporting. I also became more confident in checking the output from an AI-assisted development platform. I didn't just accept what I saw. I wanted to make sure it was correct. I also got better at solving problems. I worked on closing the gap between a prototype and a system that is ready, for real use.

About the Plant

Kota Thermal Power Plant is a thermal power station located in Kota, Rajasthan. This power station is managed by Rajasthan Rajya Vidyut Utpadan Nigam Limited. The Kota Thermal Power Plant has many pieces of equipment such as boilers, steam turbines, generators and other auxiliary equipment. All the above listed equipment should be constantly monitored and maintained daily.

While working at the Kota Thermal Power Plant for some time I became familiar with the process of equipment monitoring and maintenance there. This practical experience allowed me to think about further development of my project. Since I did not have any access to the control system of the Kota Thermal Power Plant because this is not the privilege of an intern. I decided to develop a simulation program which would simulate such process as equipment monitoring and diagnostics, but using the equipment which does not exist.

Table of Contents

1. Introduction
2. Industrial Context
3. Problem Statement
4. Objectives
5. Existing System vs. Proposed System
6. Requirements Analysis
7. Technology Stack
8. System Architecture
9. Data Model
10. Module-Wise Implementation
11. Sensor Simulation and Health Scoring
12. Testing and Validation
13. Security Analysis
14. Performance, Scalability and Maintainability
15. Engineering Decisions and Challenges
16. Limitations of the Current Implementation
17. Production Migration Plan
18. Future Scope
19. Conclusion

1. Introduction

1.1 Background

In thermal power plants, there exist big fleets of various rotating and stationary equipment like boilers, turbines, generators, pumps, cooling towers and transformers; their breakdown or wear directly influences plant reliability. Health monitoring in such power plants has been traditionally performed with the help of physical gauges, DCS screens and manual logging, which makes it hard for an engineer to get an overall picture of the status of assets.

1.2 Motivation

As an exercise in software engineering for fourth-year students, this project aimed to create software layer of such a system including dashboard, data model, workflows and visualisations, which can be used by an engineer while not requiring access to the instrumentation in the real plant. The reasons were practical training of requirements analysis, data modelling, state management and information visualisation.

1.3 Scope

Scope of the prototype built includes a browser-based single-page application that generates equipment telemetry on its own and displays it via dashboard, equipment registry, monitoring screen, fault log, maintenance tracker and report generator. The prototype does not involve any connection with IoT devices or real plant, no data from the real world; all sensor behavior is synthetic and generated by the application itself.

1.4 Target Users

- Plant Engineer - monitors equipment, logs faults, and schedules maintenance.
- Plant Administrator / Supervisor -reviews plant-wide KPIs and reports.

2. Industrial Context

The app has been set up against the background of a thermal power plant, which is mentioned as “Kota Thermal Power Plant” within the user interface. It is done only for creating a realistic setting for names of the units and their location (for example, “Boiler House - Bay 1”, “Turbine Hall - North”); otherwise, no relation exists between the app and the real power station.

2.1 Equipment Modelled

The monitored fleet spans seven equipment categories that mirror the major systems of a coal/steam thermal unit:

- Boilers - steam generation (temperature, pressure)
- Steam Turbines - mechanical conversion (RPM, vibration)
- Generators - electrical output (power, temperature)
- Cooling Pumps and Cooling Towers - heat rejection
- Transformers - power transmission
- Feed Water Pumps - boiler feed circuit

2.2 Why Software Monitoring Matters

In a real plant, any anomalies in terms of temperatures, pressures, or vibrations can be detected early enough to schedule preventive maintenance without the need for any unscheduled downtime. The current project aims to recreate the same information structure, with KPIs, trends, thresholds, alerts, and faults and maintenance workflow, but with simulated data instead of actual sensor inputs.

3. Problem Statement

Construct one web-based system for the engineer to respond to four questions for any monitored factory by looking at four things in just a couple of minutes: which assets are healthy, warning, critical, or not functioning; which notifications and faults require his attention; which maintenance is planned or overdue; and what has happened in the factory in the past 24 hours. The system should provide answers to all these questions without relying on any actual industrial sources of data.

4. Objectives

4.1 Primary Objectives

- Build a plant-wide dashboard summarising equipment counts, health distribution, and recent activity.
- Model equipment, sensors, faults, maintenance tasks and alerts as coherent, related data entities.
- Implement a believable sensor-simulation mechanism in place of real telemetry.
- Provide equipment registry, live monitoring, fault, maintenance and reporting modules.

4.2 Secondary Objectives

- Support role-based login (Engineer / Admin) with protected routes.
- Provide exportable/printable reports summarising plant health.

4.3 Learning Objectives

- Practise requirements analysis and data modelling for an unfamiliar industrial domain.
- Evaluate an AI-assisted development platform as a software engineering tool - including verifying, rather than assuming, what it actually generated.
- Understand the gap between a convincing front-end prototype and a production-ready system.

5. Existing System vs. Proposed System

“Existing system” here refers to conventional plant monitoring practice “proposed system” is this project.

Aspect	Conventional Practice	Proposed Prototype
Data view	Scattered gauges / DCS screens / log sheets	Single consolidated web dashboard
Health assessment	Manual, experience-based	Score-based, threshold-driven, colour-coded
Alerting	Manual observation	Automatic threshold notifications
Maintenance tracking	Paper / spreadsheet logs	In-app maintenance module with status and priority
Reporting	Manually compiled	Generated report view with export/print

6. Requirements Analysis

6.1 Functional Requirements

ID	Requirement
FR-1	User can log in with a role (Engineer/Admin) and access role-appropriate views.
FR-2	System displays plant-wide KPIs: total, healthy, warning, critical, and offline equipment.
FR-3	System lists all equipment with category, location, live readings, health % and status.
FR-4	System provides a live-monitoring view that refreshes simulated readings periodically.

ID	Requirement
FR-5	System logs and displays faults with severity and linked equipment.
FR-6	System tracks maintenance tasks with schedule, priority and status.
FR-7	System raises alerts/notifications when thresholds are crossed.
FR-8	System generates a report view summarising health, faults and maintenance, exportable as PDF/Excel and printable.

6.2 Non-Functional Requirements

- Usability - information should be readable at a glance via colour-coded status and progress bars.
- Responsiveness - layout should adapt across desktop and smaller viewports.
- Maintainability - typed data models and componentised UI to ease future change.
- Performance - simulated updates (every 3 seconds) should not block the UI.
- Extensibility - mock data layer should be swappable for a real backend without a UI rewrite.

7. Technology Stack

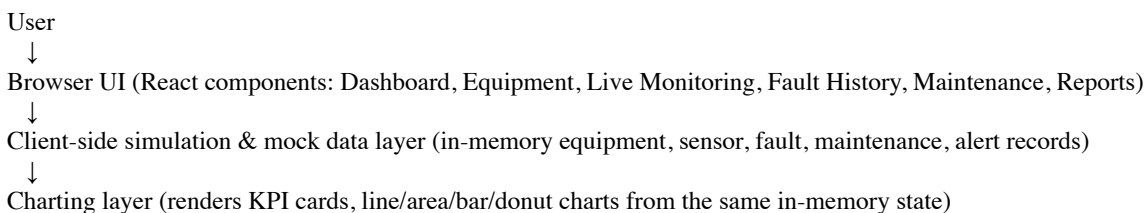
7.1 Actual Implementation (Observed in the Running Application)

Layer	Technology
Language	TypeScript / JavaScript (React component model)
UI Framework	React (component-based single-page application)
Styling	Utility-first CSS design system with a light/dark theme toggle
Charts	Client-side charting library rendering line, area, bar and donut charts
Data layer	Client-side mock/in-memory data, generated and updated in the browser
Authentication	Client-side demo login with role selection (Engineer/Admin)
Deployment	Hosted preview build served by the Lovable platform

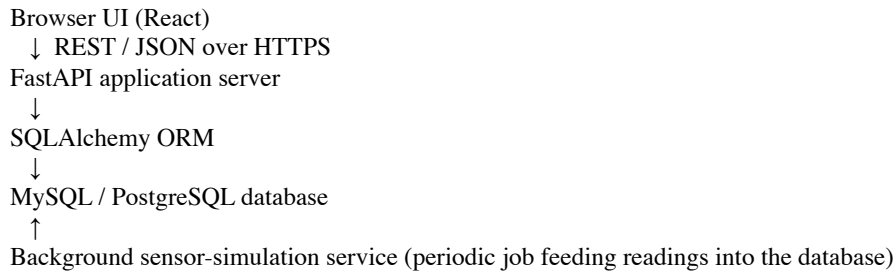
8. System Architecture

8.1 Current (Front-End-Only) Architecture

Data for all things - machines, sensors, faults, maintenance logs, and alarms - comes from the browser and gets stored in memory in the process. There is no server-based application nor database used in the prototype.



8.2 Proposed Production Architecture (Future)



9. Data Model

9.1 Equipment

Field	Example
ID	EQP-001
Name	Main Boiler A
Category	Boiler / Steam Turbine / Generator / Cooling Pump / Cooling Tower / Transformer / Feed Water Pump
Unit / Location	Unit 1, Boiler House – Bay 1
Health score	92%
Status	Healthy / Warning / Critical / Offline
Last updated	Timestamp

9.2 Sensor Readings (per equipment)

- Temperature (°C), Pressure (bar), RPM, Power (MW), Vibration (mm/s), Oil Level (%)
- Each reading is timestamped (“Last reading HH:MM:SS”) and refreshed on the live-monitoring view.

9.3 Fault

Field	Example
Type	Coolant Leakage, Bearing Overheating, Feed Water Flow Restriction
Severity	Low / Medium / High / Critical
Linked equipment	e.g. Feed Water Pump FWP-2
Date logged	e.g. 2/8/2026

9.4 Maintenance

The Maintenance module (linked from the sidebar) tracks scheduled and completed work orders; the Reports view surfaces aggregate maintenance figures (e.g. “Work orders: 16”, “Overdue maintenance: 0”). Not verified in the provided screenshots: the individual fields of a maintenance task record (assigned engineer, task type, scheduled date) were part of the original concept but were not directly captured in the supplied screenshots and should be confirmed against the live Maintenance screen before being stated as implemented.

9.5 Alert / Notification

Field	Example
Title	Boiler Temperature High
Description	Drum metal temperature crossed the warning band
Severity	High / Medium / Critical
Linked equipment	e.g. Main Boiler C
Timestamp	e.g. 02 Aug, 10:56 pm

9.6 User

A logged-in user has a name and a role (the sample session shows “Ananya Verma – Engineer”), used to personalise the header and the generated report.

9.7 Relationships

- One Equipment record has many Sensor readings (current + historical trend points).
- One Equipment record has many Faults and many Alerts.
- One Equipment record has zero or more Maintenance tasks.
- Health score and status are derived attributes of Equipment, recomputed as readings change.

10. Module-Wise Implementation

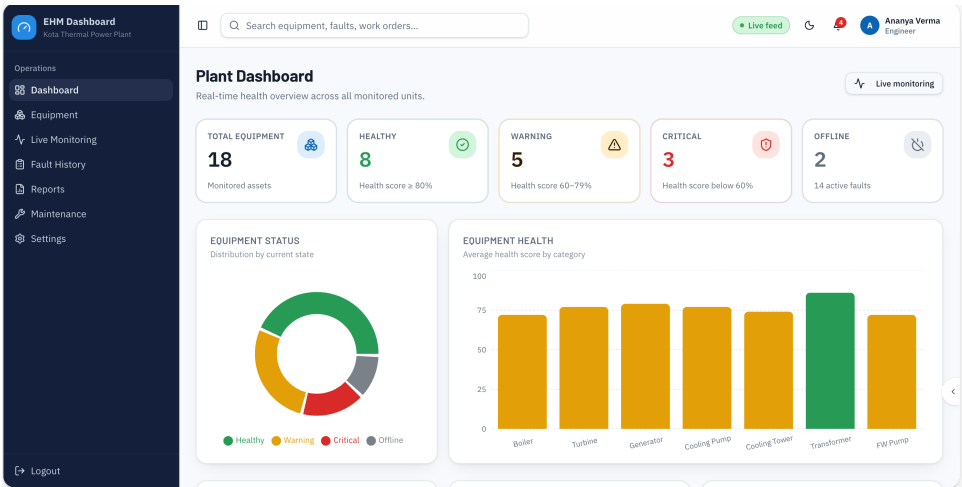
10.1 Authentication

The program demonstrates the session-based role: the title displays the username and the role (i.e., “Engineer” in the examined session), while the navigation scope consists of Operations menu that includes Dashboard, Equipment, Live Monitoring, Fault History, Reports, Maintenance, and Settings, along with a Logout button. It is in line with the client-side/mock authentication model common for an AI-developed prototype (e.g., mock user saved in the application state or local storage).

Critical limitation: this is clearly not production authentication. There is no visible trace of the server-side password hashing, token creation, and API authorization.

10.2 Dashboard

The Plant Dashboard is the first view that loads up. The first view has five KPI cards - Total Equipment, Healthy, Warning, Critical, and Offline - where every card has the threshold rules defined. The other two visual elements are the donut chart for equipment status and the health bar chart.



Scrolling further, the dashboard shows Temperature, Pressure and Power-Consumption trend charts for the last 24 hours, a Recent Alerts panel, a Recent Faults panel, and an Equipment Health Summary panel that ranks the lowest-health assets first - giving the engineer a prioritised worklist rather than a flat equipment list.

10.3 Equipment Management

The Equipment screen lists all 18 registered assets in a searchable, filterable table (search by ID/name/location/unit; filter by category and status), with live temperature, pressure, RPM, a health progress bar, status badge, last-updated timestamp, and view/edit/delete actions. An “Add Equipment” action is present in the header.

Equipment Management
18 assets registered across the plant.

Search by ID, name, location or unit:

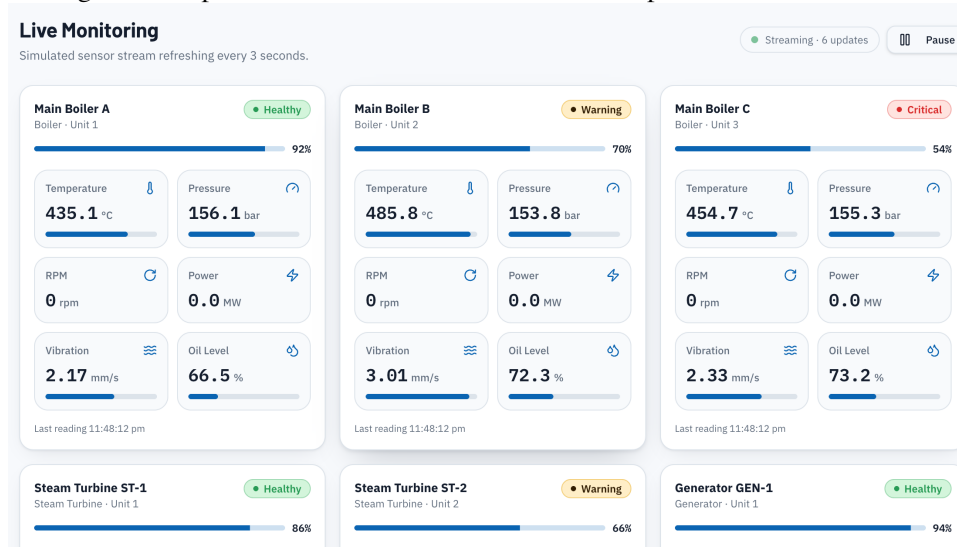
All categories:

All statuses:

ID	Equipment	Category	Location	Temp (°C)	Pressure (bar)	RPM	Health	Status	Last Updated
EQP-001	Main Boiler A Unit 1	Boiler	Boiler House - Bay 1	435.6	157.7	—	92%	Healthy	10 Jul, 11:00 pm
EQP-002	Main Boiler B Unit 2	Boiler	Boiler House - Bay 2	494.4	149.6	—	71%	Warning	10 Jul, 10:59 pm
EQP-003	Main Boiler C Unit 3	Boiler	Boiler House - Bay 3	462.1	156.3	—	54%	Critical	10 Jul, 10:58 pm
EQP-004	Steam Turbine ST-1	Steam	Turbine Hall - Bay 1	82.7	128.2	2924	88%	Healthy	10 Jul, 10:58 pm

10.4 Live Monitoring

The Live Monitoring window is clearly labeled as “Simulated sensor stream refreshing every 3 seconds,” having a streaming status, update count, and a pause button. The equipment is displayed as a card having the health bar, along with six live parameters (Temperature, Pressure, RPM, Power, Vibration, and Oil Level), with a last reading timestamp, which ensures that the simulation loop refreshes all the sensors and not just one key figure.



10.5 Fault Management and Alerts

Faults can be viewed using a special “Fault History” screen (accessed via the sidebar) and summarised on the “Recent Faults” panel of the Dashboard, each containing information about the fault type, severity, and associated equipment. Alerts can also be accessed via a bell icon in the header bar (seen with 4 unread items), each with a title, short description, severity indicator, and timestamp (example: “Boiler Temperature High - High - Main Boiler C.”)

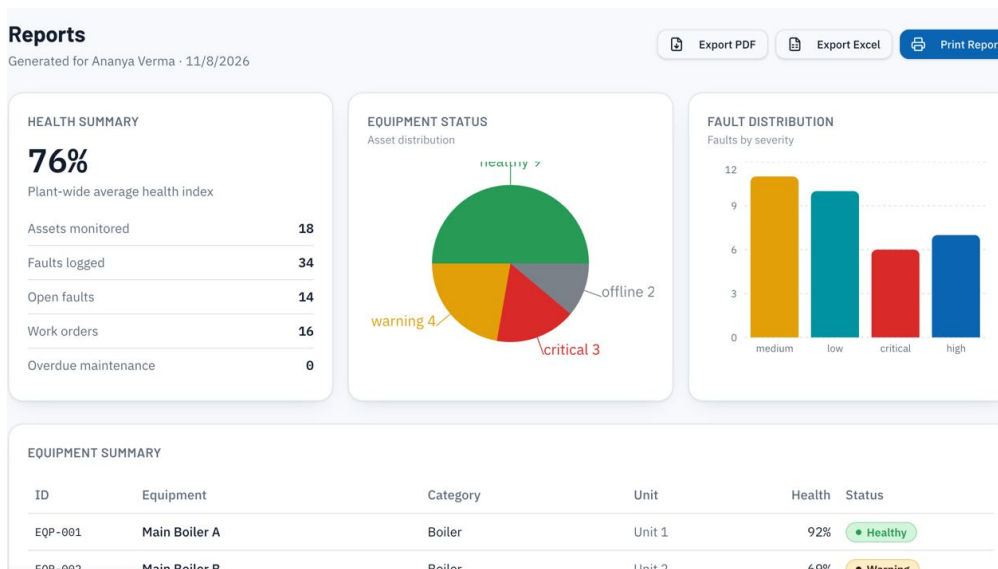
Implementation detail that is worth mentioning in a final report: within the inspected data, some alert titles and the equipment name below do not match (for example, the “Generator Overheating” alert associated with a Cooling Pump). It implies that alert messages are generated from a general set of templates rather than from a specific template corresponding to the equipment category affected by the fault - an acceptable implementation for synthetic data but needs to be changed before presenting the alerting algorithm as a realistic solution (messages generated from templates for each equipment category).

10.6 Maintenance

A Maintenance screen is present in the navigation and its aggregate figures (work orders, overdue maintenance) feed into the Reports view. Detailed field-level behaviour of this screen was not captured in the supplied screenshots and is therefore reported at a summary level only, a full field-by-field description should be added after reviewing the live Maintenance screen directly.

10.7 Reports and Export

The Reports screen generates a session-stamped summary (“Generated for Ananya Verma · 11/8/2026”) consisting of a Health Summary card (plant-wide average health index, assets monitored, faults logged, open faults, work orders, overdue maintenance), an Equipment Status pie chart, a Fault Distribution bar chart by severity, and an Equipment Summary table. Export PDF, Export Excel, and Print Report actions are provided in the header.



11. Sensor Simulation and Health Scoring

11.1 Why Simulation Instead of Static or Fully Random Data

Two naive alternatives were avoided. Static values would make the “live” views meaningless, since nothing would ever change. Fully independent random values on every refresh would make every reading jump discontinuously, which does not resemble how a real physical quantity such as drum temperature or shaft RPM actually behaves - real sensor traces move gradually around an operating point, with rare larger deviations.

11.2 Observed Behaviour

This behavior can be explained in the context of the bounded and gradual simulation where on the Live Monitoring screen each refresh (which happens every three seconds) depicts each metric as changing by some small value relative to the previous one rather than changing completely randomly to some uncorrelated value while staying within the plausible range of values for that particular equipment/metric pair (e.g. temperatures of the boilers ranging from 430 to 495°C and pressures of boilers being in the range of 143 to 169 bar).
Not found in the given material: actual formula used for updates (e.g. whether it is using explicitly the combination of a random drift, occasional jump, and normalization part).

11.3 Health Scoring

Health is represented as a 0–100% score per asset, mapped to a discrete status using fixed thresholds visible on the Dashboard KPI cards:

Status	Rule (as shown in the UI)
Healthy	Health score $\geq 80\%$
Warning	Health score 60–79%
Critical	Health score below 60%
Offline	Treated as a separate state (not score-derived), shown with its own KPI and count of active faults

12. Testing and Validation

Test Case	Steps	Expected Result	Observed Result
TC-01 Login	Open app, sign in with demo credentials	Redirected to Dashboard with user shown in header	Pass
TC-02 KPI consistency	Compare Dashboard KPI counts with Equipment table statuses	Counts match table (18 total; healthy/warning/critical/offline sum to total)	Pass
TC-03 Live refresh	Open Live Monitoring, observe two consecutive refreshes	Values change by a small, bounded amount, not an unrelated jump	Pass
TC-04 Filter/search	Filter Equipment table by category and status	Table narrows to matching rows only	Pass (as observed in UI controls)
TC-05 Alert-equipment mapping	Cross-check alert titles against tagged equipment	Alert type should match equipment category	Partial - mismatch observed,
TC-06 Report export	Trigger Export PDF / Export Excel / Print	A file downloads or print dialog opens	Not verified end-to-end in the supplied material

13. Security Analysis

- Client-side authentication: session and role appear to be held in browser state, which can be inspected or altered by any user with browser developer tools — not acceptable for production.
- No visible server-side authorization: since there is no backend, there is no API to enforce that only an Admin can, for example, delete equipment.
- No persistent storage: all data resets when the session/browser state is cleared, which limits blast radius but also means there is nothing to secure server-side yet — this becomes a real concern once a backend is added.
- No evidence of input validation being tested against a real API, since equipment creation currently writes to in-memory state only.

Recommended hardening for production: move authentication server-side with hashed credentials and issued session tokens (e.g. JWT with short expiry and refresh), enforce authorization checks on every API endpoint (not just hide UI elements), validate all inputs server-side, and serve the application over HTTPS with proper CORS configuration.

14. Performance, Scalability and Maintainability

At the current scale (18 assets, a handful of charts, a 3-second refresh interval) the client-side simulation and rendering are lightweight and responsive. This will not scale directly to hundreds of assets or sub-second telemetry, both because recomputing every chart on every tick in the browser is expensive, and because there is no server to aggregate or downsample data before it reaches the client.

- Maintainability is helped by a componentised UI (separate screens per module) and a single shared data layer that all screens read from.
- Scalability would require moving simulation and aggregation server-side, paginating/virtualising large equipment tables, and using a push mechanism (WebSockets/SSE) instead of client-side interval polling.

15. Engineering Decisions and Challenges

- Setting thresholds for health (80%/60%) to ensure KPIs and colors were easy to understand and consistent in the Dashboard, Equipment, and Reports tabs.
- Using a simulation that is bounded (or incremental), not naive random numbers, to give a feeling that the Live Monitoring tab is a true telemetry feed, not noise.
- Collaborating with an artificial intelligence-assisted development framework meant that its work product was only a first draft and had to be validated and verified.

- Striking a balance between the density of the information presented (KPIs, two charts, three lists) on the Dashboard and its readability.

• 16. Limitations of the Current Implementation

- No real backend, database, or persistence - all data is client-side and resets with the session.
- Authentication is client-side/demo only, not suitable for production use.
- Sensor data is fully synthetic; the project makes no claim to reflect real thermal-plant physics beyond plausible ranges.
- No automated test suite; verification in this report is based on manual functional testing.
- Observed inconsistency between some alert titles and their tagged equipment.
- No machine learning or predictive analytics - health scoring is threshold-based, not predictive.

17. Production Migration Plan

Step	Action
1	Stand up a FastAPI service exposing REST endpoints for equipment, sensors, faults, maintenance, and alerts.
2	Model the entities in Section 9 as SQLAlchemy models and migrate to MySQL/PostgreSQL for persistence.
3	Move sensor simulation into a background job/service that writes readings on a schedule, replacing client-side generation.
4	Replace the client-side mock data layer with API calls; keep the existing UI components unchanged where possible.
5	Implement server-side authentication (hashed passwords, issued tokens) and per-endpoint authorization by role.
6	Add push-based updates (WebSockets/SSE) for the Live Monitoring screen instead of client-side interval refresh.
7	Introduce automated tests (unit tests for health-score logic, integration tests for API endpoints).
8	Harden export functionality to generate verified PDF and true .xlsx files server-side.

18. Future Scope

- Integration with real IoT sensors/PLCs for at least one equipment category as a proof of concept.
- Predictive maintenance using historical trend data (e.g. simple regression or anomaly detection) once real or higher-fidelity historical data exists.
- Role-based dashboards tailored to Engineer vs. Admin vs. Plant Manager personas.
- Mobile-optimised or native companion app for on-floor engineers.
- Audit logging of fault acknowledgements and maintenance actions.

19. Conclusion

The Industrial Equipment Health Monitoring and Maintenance Dashboard delivers a coherent, well-organised front-end prototype covering dashboarding, live monitoring, equipment management, fault and alert handling, maintenance tracking, and reporting for a simulated thermal-plant fleet of 18 assets. Built through an AI-assisted development platform and verified by direct inspection of the running application, it demonstrates sound information architecture: KPI cards, charts, and tables all read from a single consistent data model, and health status is driven by clear, explainable thresholds rather than a black box.
